SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA43

COURSE NAME: INTERNET PROGRAMMING

Name: K. Dedeepya

Reg no: 192324252

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

<form action="/register" method="POST">

<input type="text" name="name">

<input type="email" name="email">

<button>Submit</button>

</form>

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate HTML5 semantic elements missing in the structure.

The given form is missing suitable semantic elements that improve the structure and readability of the webpage.

- <header> – Displays the title of the symposium portal.
- <main> – Contains the main registration content.
- <section> – Groups related registration information.
- <label> – Provides accessible names for input fields.
- <footer> – Displays additional information, such as college contact details.

2. Modify the structure to make it standards-compliant.

```
<!DOCTYPE html>
<html>
<head>
  <title>Symposium Registration</title>
</head>
```

```
<body>
  <h1>Online Symposium Registration</h1>
  <h2>Student Registration</h2>
  <form>
    Name: <input type="text" required><br><br>
    Email: <input type="email" required><br><br>
    <button type="submit">Submit</button>
  </form>
  <footer>College Symposium Portal</footer>
</body>
</html>
```

Explanation: The document includes the HTML5 doctype, language attribute, character encoding, viewport settings, semantic elements, associated labels, and a submit button. The required attribute ensures that the student enters both fields before submission.

3. Interpret the HTTP request generated during form submission.

When the student submits the form, the browser sends an HTTP POST request to then server.

Example:

```
POST /register HTTP/1.1
Host: college.example
Content-Type: application/x-www-form-urlencoded
name=Arun&email=arun%40example.com
```

Interpretation:

- `POST` submits the registration information.
- `/register` identifies the server endpoint.
- `Content-Type` specifies the format of the submitted data.
- The request body contains the student's name and email address.

4. Analyse the HTTP response cycle from server to browser.

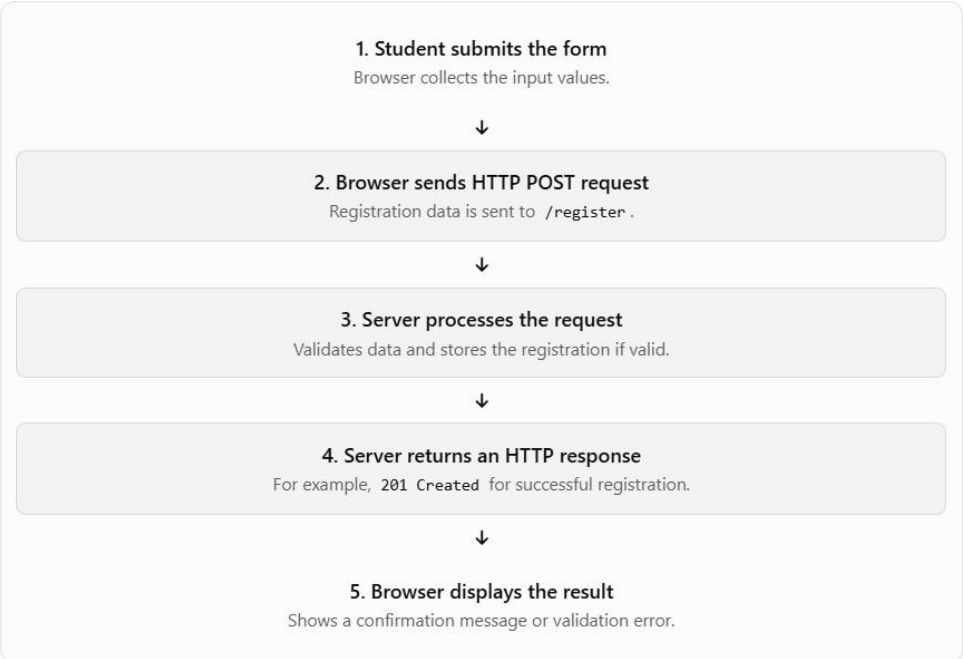
After receiving the request, the server validates the information and processes the registration. It then returns an HTTP response.

Example HTTP Response

```
HTTP/1.1 201 Created
Content-Type: text/html
```

The response body could contain a registration confirmation page.

A successful response indicates that the requested operation was completed. If the submitted data is invalid, the server should return an appropriate error response.



5. Suggest improvements for usability and accessibility.

- Use labels associated with inputs through the for and id attributes.
- Add required attributes and appropriate input types for validation.
- Display clear success and error messages after submission.
- Use readable fonts, suitable colours, and a responsive layout.
- Provide keyboard navigation and visible focus indicators.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used: <html>

<head>

<title>Company</title>

</head>

<body>

<h1>Welcome

<p>About us

</body>

</html>

Questions:

1. Identify improper nesting and missing closing tags.

** **

<h1>Welcome

** About us**

** **

Element	Problem	Correction
<html>	Language not specified	Add lang="en"
<head>	Character encoding not specified	Add <meta charset="UTF-8">
<h1>	Missing explicit closing tag	Add </h1>
** **	Missing explicit closing tag	Add
** **	Generic container without semantic meaning	Use suitable semantic elements where appropriate

2. Analyse how different browsers may interpret this structure.

1. Browsers may automatically close certain elements when they encounter new elements or the end of the document.
2. The resulting DOM structure may not match the developer's intended structure.
3. Default heading and paragraph margins can affect the page layout.
4. Differences in browser defaults, fonts, and CSS rendering can cause visual inconsistencies.
5. Invalid or ambiguous markup makes debugging and maintenance more difficult.

Analysis: Although modern browsers often display malformed HTML, correcting the source ensures a predictable document structure and reduces compatibility problems.

3. Identify missing semantic elements (e.g., header, section).

The following HTML5 semantic elements can improve the webpage structure:

- <header> – Contains the company name or page heading.
- <main> – Contains the primary page content.
- <section> – Groups related content, such as the About Us section.
- <footer> – Contains copyright or contact information.

4. Provide a corrected W3C-compliant HTML structure.

<!DOCTYPE html>

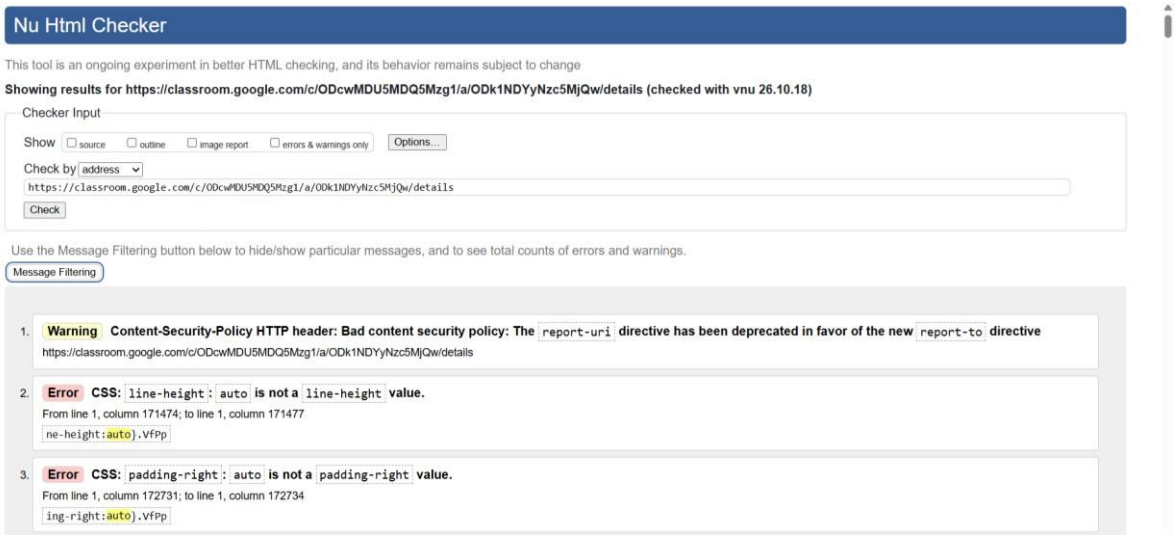
<html>

<head>

<title>Company Website</title>

```
</head>
<body>
  <header><h1>Welcome</h1></header>
  <main>
    <h2>About Us</h2>
    <p>Learn about our company and services. </p>
  </main>
  <footer>&copy; 2026 Company Website</footer>
</body>
</html>
```

Explanation: This code uses valid HTML5 syntax, correct element nesting, closing tags, semantic elements, and essential metadata. The document follows standard HTML structure and can be checked using the W3C-compatible Nu HTML Checker.



5. Suggest techniques to ensure cross-browser compatibility.

- Validate HTML using the Nu HTML Checker.
- Use standard HTML5 elements and valid CSS properties.
- Add a CSS reset or normalize stylesheet to reduce inconsistent browser defaults.
- Test the webpage in Chrome, Firefox, Edge, and Safari where available.
- Use responsive CSS with Flexbox, Grid, and media queries.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

<form action="/submit" method="GET">

<input type="text" name="username">

<input type="password" name="password">

<button type="button">Login</button>

</form>

Questions:

1. Identify issues in the form configuration.

```
<form action="/submit" method="GET">
   <input type="text" name="username">
   <input type="password" name="password">
   <button type="button">Login</button>
</form>
```

Component	Issue	Correction
method="GET"	Unsuitable for transmitting passwords	Use POST
type="button"	Does not submit the form automatically	Use type="submit"
Input fields	No labels or required validation	Add labels and required
Security	GET can expose passwords in the URL	Use POST over HTTPS

2. Analyse why the HTTP request is not triggered.

The main reason is the button configuration.

<button type="button">Login</button>

A button with type="button" does not submit the form automatically. Therefore, clicking it does not trigger the normal HTML form-submission process.

To submit the form, use: **<button type="submit">Login</button>**

When the submit button is clicked, the browser validates applicable form fields and sends a request if validation succeeds.

3. Interpret the role of GET method in this scenario.

The GET method retrieves information from a server. When used with an HTML form, the browser normally appends the submitted form data to the URL as query parameters.

For example: **/submit?username=Dedeepya&password=1234**

This is unsuitable for login credentials because passwords may appear in browser history, server logs, or other URL-handling systems.

The GET method is appropriate for non-sensitive searches and filtering, while POST is generally appropriate for submitting login credentials in the request body. POST alone does not encrypt data; HTTPS is necessary to protect it during transmission

4. Propose a corrected implementation.

```
<!DOCTYPE html>
<html>
<head>
```

```
    <title>Login Page</title>
</head>
<body>
    <h1>User Login</h1>
    <form>
        Username: <input type="text" required><br><br>
        Password: <input type="password" required><br><br>
        <button type="submit">Login</button>
    </form>
</body>
</html>
```

Explanation: The corrected code uses method="POST", a submit button, associated labels, and required fields. The server endpoint /submit must be configured to process POST requests, verify credentials securely, and return the appropriate response.

5. Suggest improvements for secure data transmission.

- Use HTTPS to encrypt communication between the browser and server.
- Use POST rather than GET to avoid placing passwords in the URL.
- Validate and sanitize input on the server to prevent injection attacks.
- Store passwords using a secure password-hashing algorithm such as Argon2id or bcrypt, never as plain text.
- Apply rate limiting and account protection to reduce brute-force login attempts.